

บทที่ 2

การพัฒนาโปรแกรมด้วยวิธีเชิงวัตถุและยูเอ็มแอล

2.1 บทนำ

ในราวๆปี 1990 มีวิธีที่ใช้ Model ระบบมากมาย แต่มีอยู่ 3 วิธีที่เป็นที่นิยมใช้ในการ model ระบบคือ OMT (Rumbaugh), Booch และ OOSE (Jacobson) โดยในแต่ละวิธี จะมีข้อดีข้อเสียต่างกัน วิธีแบบจำลองรูปภาพเชิงวัตถุ (Object Modeling Technology: OMT) นั้นจะมีจุดแข็งในการวิเคราะห์ระบบได้ง่าย แต่พื้นที่ออกแบบนั้นจะใช้อยาก ส่วนวิธี Booch นั้นจะมีจุดแข็งในการออกแบบ แต่การวิเคราะห์จะยาก และวิธีสุดท้ายวิธีแบบจำลองรูปภาพเชิงวัตถุทางวิศวกรรมซอฟต์แวร์ (Object Oriented Software Engineering: OOSE) จะมีจุดแข็งในคุณสมบัติของการวิเคราะห์แต่มีจุดอ่อนในจุดต่างๆอีก เมื่อเวลาผ่านไป การใช้วิธีนำเสนอรายละเอียดงานที่ต่างกัน ทำให้เกิดความสับสน ดังนั้นจึงจำเป็นต้องมีวิธีที่มาตรฐานที่สามารถเข้าใจได้ง่าย และยังคงอยู่บนพื้นฐานของวิธีเดิมที่มีอยู่ UML จึงได้ถือกำเนิดขึ้นมาจากการร่วมมือกันของทั้ง Booch ,Jacobson ,Rumbaugh รวมถึงคนอื่น โดยได้นำข้อดีของแต่ละวิธีมารวมกัน

Unified Model Language หรือ UML เป็นภาษาโมเดลที่นำเอาสัญลักษณ์มาช่วยในการเพิ่มประสิทธิภาพในส่วนของการวิเคราะห์(Analysis) และออกแบบ(Design) ซึ่งมีประโยชน์ในหลายด้าน โดยจะเป็นรูปแบบของออปเจกต์โอเรียนเท็ด ซึ่งการออกแบบจะทำให้เราสามารถวิเคราะห์ส่วนประกอบโดยรวมของระบบและยังสามารถที่จะรองรับภาษาที่จะใช้ในการพัฒนาได้หลากหลายภาษามาก และยังหลีกเลี่ยงความซ้ำซ้อนและเฉพาะเจาะจงเกินไป ซึ่งการออกแบบโดยวิธี UML นี้สามารถที่จะแสดงออกมาในรูปแบบของไดอะแกรม(Diagram)ต่างๆ

แอปพลิเคชันขั้นสูงที่พัฒนาขึ้นนั้นต่างก็มีขนาด ความซับซ้อน และการกระจายที่มากขึ้น การพัฒนาซอฟต์แวร์ให้สามารถทำงานได้ทั้งเร็ว และถูกต้องนั้น จึงจำเป็นต้องมีการพัฒนาร่วมกันเป็นทีม ซึ่งความยากอย่างหนึ่งของทีมพัฒนาซอฟต์แวร์ก็คือการสื่อสารระหว่างนักพัฒนาในทีม เนื่องจากเมื่อซอฟต์แวร์มีขนาดใหญ่ นักพัฒนาในทีมก็มีจำนวนมากขึ้นตามขนาดของซอฟต์แวร์ และแต่ละท่านก็มีความเชี่ยวชาญในด้านต่างๆกัน นอกจากนั้นเทคโนโลยีในการพัฒนามีการเปลี่ยนแปลงที่รวดเร็วมาก

นอกจากนั้น ปัญหาของการพัฒนาซอฟต์แวร์ ส่วนหนึ่งเกิดจากการที่นักพัฒนาไม่สามารถจัดการกับความต้องการที่เปลี่ยนแปลงของแต่ละโมดูล รวมถึงซอฟต์แวร์ที่พัฒนาขึ้นนั้นไม่สามารถทำงานร่วมกันได้ และยังดูแลรักษาและเปลี่ยนแปลงได้ยาก นักพัฒนาในทีมไม่สามารถติดตามได้ว่าใครแก้ไขอะไร เมื่อใด ที่ไหน และทำไม

สาเหตุสำคัญที่ทำให้เกิดปัญหาในการพัฒนาซอฟต์แวร์ก็คือ ไม่มีระบบจัดการความต้องการที่ดีเพียงพอ มีการสื่อสารที่ไม่รัดกุมและกำกวม สถาปัตยกรรมของซอฟต์แวร์ไม่มั่นคง

ดังนั้น การจัดการกระบวนการในการพัฒนาจึงจำเป็นต้องการพัฒนาซอฟต์แวร์ให้ถูกต้อง และมีคุณภาพตามที่ต้องการ จึงจำเป็นต้องอย่างยิ่งที่จะต้องศึกษาการพัฒนาระบบด้วยวิธีเชิงวัตถุ, กระบวนการพัฒนาซอฟต์แวร์

ที่ดี, ส่วนประกอบของยูเอ็มแอล และเครื่องมือที่ใช้วิเคราะห์และออกแบบซึ่งใช้ Rational Rose เพื่อการพัฒนา ระบบซอฟต์แวร์ที่มีคุณภาพตามความต้องการมากที่สุด

2.2 การพัฒนาระบบด้วยวิธีเชิงวัตถุ

การพัฒนาระบบด้วยวิธีเชิงวัตถุนั้นเป็นการพัฒนาที่ช่วยลดความยุ่งยากซับซ้อนของระบบใหญ่ๆได้ โดยผู้พัฒนาจะมองระบบและส่วนประกอบหรือระบบย่อย (Sub System) เป็นวัตถุ (Object) เนื่องจากสิ่งต่างๆ ที่มีอยู่ในโลกความเป็นจริงนั้นก็ถือได้ว่าเป็นวัตถุได้อยู่แล้ว และด้วยการมองระบบเป็นวัตถุทำให้ผู้พัฒนาระบบสามารถมองระบบได้ชัดเจนมากยิ่งขึ้น ส่งผลให้การวิเคราะห์และออกแบบระบบทำได้ง่ายและมีความถูกต้องมากขึ้น

การพัฒนาระบบด้วยวิธีเชิงวัตถุนั้น ยังมีส่วนเกี่ยวข้องกับทุกขั้นตอนของการพัฒนาระบบ ทั้งการรวบรวมความต้องการ (Requirement gathering), การวิเคราะห์ (analysis), การออกแบบ (design), การเขียนโปรแกรม (coding), และการทดสอบ (testing) ซึ่งเป็นการแก้ปัญหาของการพัฒนาระบบสมัยก่อนแบบที่เรียกว่า การพัฒนาแบบโครงสร้าง (Structural) ซึ่งเป็นการบอกเพียงว่าระบบจะมีลักษณะการทำงานอย่างไรแต่ไม่ได้บอกถึงวิธีการพัฒนาระบบหรือรูปแบบการเขียนโปรแกรมเลย จุดนี้เองทำให้หลายๆครั้ง ระบบที่พัฒนาขึ้นมีปัญหาเนื่องจากการตรวจสอบความถูกต้องของระบบ (Validate) ระบบในเชิงพฤติกรรม (Behavior) นั้นทำได้ยาก

2.3 ข้อดีของการพัฒนาระบบด้วยวิธีเชิงวัตถุ

2.3.1 Higher level of abstraction วิธีการวิเคราะห์ระบบแบบโครงสร้างนั้นมีข้อจำกัดในการแยกแยะเอกลักษณ์ และการซ่อนรายละเอียด อย่างไรก็ตามการวิเคราะห์และพัฒนาระบบแบบโครงสร้างนี้ จะแยกแยะโครงสร้างและการทำงานของระบบออกจากกัน เพื่อให้เกิดประสิทธิภาพในการทำงาน ตัวอย่างเช่น เซนเซอร์ จะได้รับการโมเดลให้เป็นส่วนหนึ่งของข้อมูลการอ่านหรือการกระทำอื่นๆ กับเซนเซอร์จะถือว่าเป็นโอเปอเรชัน (Operation) แต่การโมเดลนี้ไม่ได้แสดงความสัมพันธ์ระหว่างข้อมูลและโอเปอเรชัน

ส่วนโมเดลของการพัฒนาเชิงวัตถุนั้นจะแสดงความสัมพันธ์ระหว่างข้อมูลกับโอเปอเรชันที่กระทำกับข้อมูลอย่างใกล้ชิด เนื่องจากโมเดลนี้จำลองมาจากโลกจริงๆ การแยกแยะเอกลักษณ์ของการพัฒนาเชิงวัตถุนั้น ให้ความเข้าใจและความสามารถที่ดีกว่า รวมทั้งมีการตั้งชื่อสำหรับวัตถุนั้นมาจากชื่อที่มีอยู่จริงในโดเมนของปัญหา มุมมองของวัตถุจะเป็นการมองในระดับสูงและใกล้เคียงกับโดเมนของปัญหามากกว่า

2.3.2 Seamless transition among different phases of software development การพัฒนาซอฟต์แวร์นั้นต้องการวิธีการ (Methodology) ที่แตกต่างกันในแต่ละช่วงของการพัฒนา การเปลี่ยนแปลงจากช่วงการทำงานหนึ่งไปช่วงอื่นๆในแต่ละ Process จะมีความซับซ้อน และยังไม่มีความสอดคล้องกัน และยังซ้ำด้วย และยังทำให้ขนาดของงานใหญ่ขึ้น และมีโอกาสที่จะเกิดความผิดพลาดขึ้นด้วย แต่วิธีการของเชิงวัตถุนั้นจะใช้ภาษาเดียวกันในการติดต่อระหว่างการวิเคราะห์ การออกแบบ การเขียนโปรแกรม และการออกแบบฐานข้อมูลด้วย ทำให้ลดระดับของความซับซ้อนลง และช่วยให้การเข้าใจระบบก็ชัดเจนด้วย

2.3.3 Encouragement of good programming techniques เป็นการสนับสนุนเทคนิคการเขียนโปรแกรมที่ดี เพราะการเขียนโปรแกรมเชิงวัตถุ นั้นจะเป็นการเขียนในลักษณะของคลาส ซึ่งบอกว่าคลาสนี้ทำอะไร และ

ทำอะไร และคุณสมบัติของระบบนั้นสามารถที่จะรวมคลาสเข้าด้วยกันเป็นระบบย่อยได้ และการเปลี่ยนแปลงคลาสหนึ่งก็จะไม่มีผลกระทบต่อคลาสอื่นๆด้วย อย่างไรก็ตามภาษาต่างๆเช่น C++, Smalltalk หรือ Java ก็เพิ่มส่วนที่สามารถรองรับการออกแบบเชิงวัตถุด้วยเพื่อง่ายต่อการสร้าง และการนำ Code มาใช้ใหม่กับแนวคิดของคลาสและการสืบทอด (Inheritance)

2.3.4 Promotion of Reusability ระบบแบบโครงสร้างมีข้อจำกัดในการนำสิ่งที่พัฒนาไปแล้วกลับมาใช้ใหม่ แต่ในความเป็นจริงแล้ว ถ้าผู้พัฒนาระบบมีคอมโพเนนต์ อยู่แล้วก็ควรจะใช้ได้เลย แต่ระบบแบบโครงสร้างนี้ผู้ที่นำมาใช้ต้องทำการเปลี่ยนแปลงซอร์สโค้ด (Source Code) เพื่อให้เหมาะสมกับความต้องการของระบบใหม่เพื่อที่จะใช้งานได้ แต่การพัฒนาเชิงวัตถุนี้มีคุณสมบัติที่สนับสนุนการนำกลับมาใช้ใหม่โดยตรงคือ Generalization และ Refinement โดย Generalization คือการสืบทอดคุณสมบัติโดยจะได้คุณสมบัติเดิมทุกประการและยังสามารถเพิ่มและขยายคุณสมบัติได้ด้วย โดยไม่ต้องมีการเปลี่ยนแปลงซอร์สโค้ดเก่าเลย ลักษณะการนำมาใช้ใหม่นี้เรียกว่า การเขียนโปรแกรมแบบแตกต่าง ส่วน Refinement นั้นก็คล้ายๆกับ Generalization แต่จะยอมให้นักพัฒนาใช้วัตถุที่ไม่สมบูรณ์โดยการเพิ่มส่วนที่หายไป ตามที่ตนเองต้องการได้

2.3.5 Promotion of Scalability ระบบโครงสร้างมีข้อจำกัดในการปรับเปลี่ยนขนาดของระบบ ทั้งนี้เนื่องจากระบบแบบโครงสร้างมี การแยกแยะเอกลักษณ์และการซ่อนรายละเอียดที่ไม่ดี ทำให้ระบบแบบโครงสร้างมีความซับซ้อนมากขึ้นเมื่อมีปัญหาที่เกิดขึ้นกับระบบมากขึ้น แต่ระบบที่พัฒนาด้วยวิธีเชิงวัตถุ นั้นจะแยกแยะเอกลักษณ์และการซ่อนรายละเอียดส่งที่ดีส่งผลให้คอมโพเนนต์แต่ละส่วนแยกออกจากกันอย่างชัดเจน และการใช้เครื่องหมาย (Notation) เดียวกันตลอดการพัฒนา ทำให้สามารถเปลี่ยนจากการวิเคราะห์เป็นการออกแบบได้ง่าย การปรับเปลี่ยนระบบให้มีขนาดใหญ่ขึ้นก็สามารถทำได้ง่ายตามด้วย

2.3.6 Promotion of Reliability and Safety เนื่องจากการพัฒนาเชิงวัตถุมีการแยกแยะเอกลักษณ์และการซ่อนรายละเอียดที่ดีกว่าทำให้การสื่อสารระหว่างวัตถุที่ไม่เกี่ยวข้องกันจำกัดอยู่ด้วยอินเทอร์เฟซ ที่ผู้พัฒนาได้กำหนดไว้เท่านั้น ส่งผลให้ระบบที่พัฒนาด้วยวิธีเชิงวัตถุมีความน่าเชื่อถือได้มากกว่าเพราะว่าผู้พัฒนาสามารถควบคุมการติดต่อระหว่างคอมโพเนนต์ได้ และยังสามารถเพิ่มเงื่อนไขก่อนหน้าและท้าย (Pre and Post Condition) เพื่อให้ระบบสามารถทำงานได้อย่างถูกต้อง

2.3.7 Promotion of Concurrency การทำงานไปพร้อมๆกัน (Concurrency) เป็นรูปแบบการทำงานของสิ่งที่มีตัวตนในโลกแห่งความเป็นจริง และยังเป็นคุณสมบัติสำคัญของคอมพิวเตอร์ยุคใหม่ด้วย การพัฒนาแบบโครงสร้างไม่มีสัญลักษณ์สำหรับการทำงานพร้อมกัน การจัดการงาน (Task) หรือการซิงโครไนเซชัน (Synchronization) ระหว่างงานดังนั้น โครงสร้างที่สำคัญๆของระบบอาจจะไม่สามารถพัฒนาขึ้นได้โดยใช้รูปแบบพัฒนาแบบโครงสร้างมาตรฐานได้ แต่การพัฒนาแบบวิธีเชิงวัตถุสนับสนุนการทำงานพร้อมกันในตัวเอง และรายละเอียดของงาน การทำ Synchronization ระหว่างงานก็สามารถแสดงออกมาในแผนภาพ (Statecharts) ของวัตถุที่ทำงานแบบแอคทีฟ (Active Object) และการส่งแมสเสจ (Message) ระหว่างวัตถุ แผนภาพเหล่านี้เป็นเครื่องมือที่มีความเหมาะสมสำหรับการสร้างระบบที่มีเงื่อนไขทางด้านประสิทธิภาพ

2.4 ความสามารถของคลาส

2.4.1 ลำดับชั้นของคลาส Class Hierarchy

2.4.1.1 Inheritance คือการสืบทอดคุณสมบัติเป็นการสร้างคลาสใหม่ขึ้นมา โดยมีพื้นฐานมาจากคลาสเดิม แต่จะมีข้อมูล (Data) หรือเมธอด ที่พิเศษเป็นของตัวเองเพิ่มขึ้นมาจากคลาสพื้นฐานเดิม ซึ่งก่อให้เกิดการแตกสายพันธุ์ของวัตถุ ขึ้นเป็นวัตถุใหม่ที่มีคุณสมบัติของวัตถุเดิมซ่อนอยู่ไม่มากนักนั่นเอง

ปัญหาสำคัญของการสืบทอดคุณสมบัติ อยู่ตรงที่ว่า ในการสร้างคลาสขึ้นมาใหม่แต่ละครั้งนั้น คลาสเหล่านั้นมีอะไรที่เป็นพื้นฐานเหมือนกันและมีคุณสมบัติอะไรที่แตกต่างกัน นักพัฒนาจะจำแนกคลาสเหล่านั้นได้อย่างไร จึงจะเหมาะสมกับพฤติกรรมการทำงานของมัน คำตอบก็คือ จะต้องเริ่มต้นมองที่ยอดบนของลำดับชั้นของคลาสดีก่อน คลาสที่อยู่สูงที่สุดจะต้องเป็นคลาสที่มีคุณสมบัติพื้นฐาน (Simplest) ของคลาสต่างๆ ให้มากที่สุด แล้วค่อยๆ แบ่งลำดับชั้นออกไปตามคุณสมบัติอื่นๆ ที่รองลงไป โดยเมื่อคุณสมบัติอย่างหนึ่งที่กำหนดลงในคลาสพื้นฐานแล้ว ทุกๆ คลาสที่สืบทอดคุณสมบัติภายใต้คลาสนั้นจะได้รับคุณสมบัติติดตัวไปด้วยอัตโนมัติ

2.4.1.2 Multiple Inheritance หรือ MI เป็นการสร้างคลาสขึ้นมาโดยการสืบทอดคุณสมบัติมาจากคลาสพื้นฐานที่มีมากกว่า 1 ตัว ทำให้ภาษาในลักษณะของการเขียนโปรแกรมเชิงวัตถุมีความยืดหยุ่นในการทำงานเพิ่มมากขึ้นอีกด้วย

2.4.2 แพ็กเกจ (Packages) เป็นการรวมคลาสที่มีความสัมพันธ์ใกล้ชิดกันเข้าไว้ในแพ็กเกจ เพื่อประโยชน์สำหรับระบบที่มีคลาสเป็นจำนวนมาก ในระบบที่มีความซับซ้อนเราอาจจะสร้างแพ็กเกจขึ้นมาก่อน แล้วจึงบรรจุคลาสและความสัมพันธ์เข้าไว้ในแพ็กเกจ

2.4.3 Encapsulation and Information Hiding เอ็นแคปซูลชัน คือ การรวมกันของโครงสร้างข้อมูล (Data Structure) กับฟังก์ชันที่เกี่ยวข้องหรือเรียกใช้ข้อมูลนั้น (เรียกว่าเมธอด หรือ Action) เกิดเป็นวัตถุตัวใหม่ที่สามารถทำการซ่อนข้อมูลจากภายนอกได้ ซึ่งจะเรียกวัตถุนั้นว่า คลาส นั่นเอง

ในการพัฒนาโดยวิธีแบบโครงสร้างในสมัยก่อนนั้น ส่วนที่เป็นโครงสร้างข้อมูลและฟังก์ชันที่เรียกใช้ข้อมูลนั้นจะถูกสร้างขึ้นแยกจากกัน นั่นคือจะมองข้อมูลและฟังก์ชันเป็นโมดูลเดี่ยวๆ ไม่มีความสัมพันธ์กันแต่อย่างใด ข้อมูลชุดหนึ่งๆ อาจมีฟังก์ชันหลายๆ ตัวมาเรียกใช้งานโดยตรง ซึ่งแต่ละฟังก์ชันอาจจัดการกับโครงสร้างของข้อมูลไม่เหมือนกัน การเข้าถึงข้อมูลโดยตรงของฟังก์ชันเหล่านี้จะนำไปสู่ปัญหาต่อไป

เอ็นแคปซูลชันก้าวมาในจุดนี้ ผลจากการที่นำโครงสร้างข้อมูลและฟังก์ชันที่ใช้งานข้อมูลนั้นมารวมกันสร้างความสัมพันธ์ให้กันและกัน จะทำให้ข้อมูลมีความมั่นคงขึ้น โดยการจะเข้ามาใช้ข้อมูลจะต้องเรียกผ่าน เมธอดเท่านั้น ผลก็คือ ไม่มีการเข้าถึงข้อมูลโดยตรงอีกต่อไป ซึ่งนั่นก็คือ เป็นการทำ Data Hiding นั่นเอง

ข้อดีอีกประการหนึ่งของเอ็นแคปซูลชัน ก็คือ เราสามารถแบ่งระดับของการเข้าถึงข้อมูลได้โดยตัวของเมธอดของวัตถุเอง

2.4.4 Polymorphism โพลิมอร์ฟิซึม หมายถึงการที่คลาสหนึ่งๆ สามารถเปลี่ยนไปได้หลายรูปร่างขึ้นอยู่กับสภาพแวดล้อมหรือสถานการณ์ในขณะนั้น

โดยจะเป็นการยอมให้ผู้พัฒนาใช้เมธอดชื่อเดียวกัน แต่การทำงานไม่เหมือนกัน จำแนกไปตามชนิดของข้อมูลหรือเมสเสจ(Message) ที่เมธอดจะได้รับขณะทำงาน เมื่อได้รับข้อมูลที่ต้องการแล้วก็จะไปเลือกเมธอดที่เหมาะสมที่สุดกับข้อมูลนั้นๆ ไปทำงานเองต่อไป

ความสามารถอีกอย่างหนึ่งของโพลิมอร์ฟิซึมก็คือ สามารถกำหนดการดำเนินการใหม่ให้กับตัวดำเนินการหรือเมธอดได้ เรียกว่าการทำ " Overloading " ยังผลให้เราสามารถเรียกชื่อเมธอดหรือตัวดำเนินการนั้น มาใช้งานกับวัตถุต่างๆ ในกรณีที่แตกต่างกัน ขึ้นอยู่กับว่าในขณะนั้น โปรแกรมกำลังทำงานอยู่กับวัตถุอะไร

2.4.5 ความสัมพันธ์

2.4.5.1 Generalization เป็นความสัมพันธ์แบบการให้กำเนิดคลาสย่อย (Subclass) จากคลาสแม่ (Superclass) โดยคลาสย่อยจะสืบทอดแอตทริบิวต์, โอเปอเรชันและความสัมพันธ์ทั้งหมดจากคลาสแม่ ทั้งของตัวเองและจากคลาสแม่ของคลาสแม่ขึ้นไปเรื่อยๆ คลาสย่อยอาจมีการเพิ่มแอตทริบิวต์และโอเปอเรชันในคลาสของมันเอง ความสัมพันธ์แบบนี้อาจถูกเรียกได้ว่า is-a หรือ kind-of

2.4.5.2 Aggregation เป็นความสัมพันธ์แบบเป็นส่วนประกอบหรือที่เรียกว่า Part-of คำว่าเป็นส่วนประกอบบางครั้งอาจมีความหมายที่ต่างกัน เช่น เครื่องยนต์และล้อเป็นส่วนประกอบของรถยนต์ในขณะนี้เรากำลังขับรถอยู่แต่ถ้ารถยนต์คันนั้นอยู่ในระหว่างการซ่อมที่อู่ซ่อมรถ นั่นคือเครื่องยนต์และล้อก็ไม่ได้เป็นส่วนประกอบของรถยนต์แล้ว

2.4.5.3 Association เป็นความสัมพันธ์แบบสองทิศทาง ซึ่งใช้ในการติดต่อระหว่างคลาส ความสัมพันธ์แบบ แอสโซซิเอชันระหว่างคลาสหมายถึง การเชื่อมต่อระหว่างออบเจกต์ในคลาสทั้งสอง ที่ปลายของเส้นความสัมพันธ์จะมีการระบุชื่อบทบาทเพื่อบอกว่าคลาสนั้นถูกมองว่าเป็นอย่างไรจากคลาสอื่น

2.4.5.4 Constraints เป็นการกำหนดข้อบังคับของความสัมพันธ์ระหว่างเอนติตี้ โดยเอนติตี้จะหมายถึงออบเจกต์, คลาส, บทบาท, แอตทริบิวต์, การเชื่อมต่อ (Link) และความสัมพันธ์ ข้อบังคับนี้จะเป็นการจำกัดค่าที่เอนติตี้สามารถเป็นไปได้

2.5 Object Oriented Methodologies

วิธีการพัฒนาระบบเชิงวัตถุนั้นมีมากมายหลายวิธี โดยแต่ละวิธีก็มีจุดเด่นและจุดด้อย แตกต่างกันไป โดยผู้พัฒนาควรจะใช้วิธีการที่มีประสิทธิภาพที่ดีที่สุด เพื่อระบบที่ดีที่สุด โดยจะใช้วิธีผสมนำข้อดีของแต่ละวิธีมาใช้ก็ได้ดังเช่น Unified Modeling Language (UML) ที่นำข้อดีของ Rumbaugh et al., Booch และ Jacobson มาใช้ และยังยังสามารถใช้เทคนิคต่างๆ ที่เกิดขึ้นมาพัฒนาด้วยเช่น Patterns และ Frameworks เป็นต้น

2.5.1 Rumbaugh et al.'s Object Modeling Technique (OMT) ถูกนำเสนอโดย Jim Rumbaugh และผู้ร่วมงาน โดยจะบอกถึงวิธีการวิเคราะห์ ออกแบบ และ ลงมือทำ ของระบบโดยวิธีเชิงวัตถุ OMT จะมีรายละเอียดต่างๆ เช่น คลาส, แอตทริบิวต์, เมธอด, การสืบทอดคุณสมบัติและแอสโซซิเอชันสามารถแสดงได้อย่างง่าย โดย OMT จะประกอบไปด้วย 4 ขั้นตอนต่อไปนี้ และในแต่ละขั้นตอนสามารถกลับมาทำซ้ำอีกหลายครั้งได้

- การวิเคราะห์ (Analysis)
- ออกแบบระบบ (System design)
- ออกแบบวัตถุ (Object design)
- การสร้าง (Implementation)

OMT แบ่งแยก model ต่างๆออกเป็น 3 ส่วนดังต่อไปนี้คือ

- Object model สามารถนำเสนอได้โดยใช้โมเดลวัตถุและการอธิบายข้อมูล(Data dictionary)
- Dynamic model สามารถนำเสนอได้โดยใช้สเตทไดอะแกรมและอีเวนทโฟลไดอะแกรม
- Functional model สามารถนำเสนอได้โดยใช้ลำดับโฟลไดอะแกรมและข้อบังคับต่างๆ

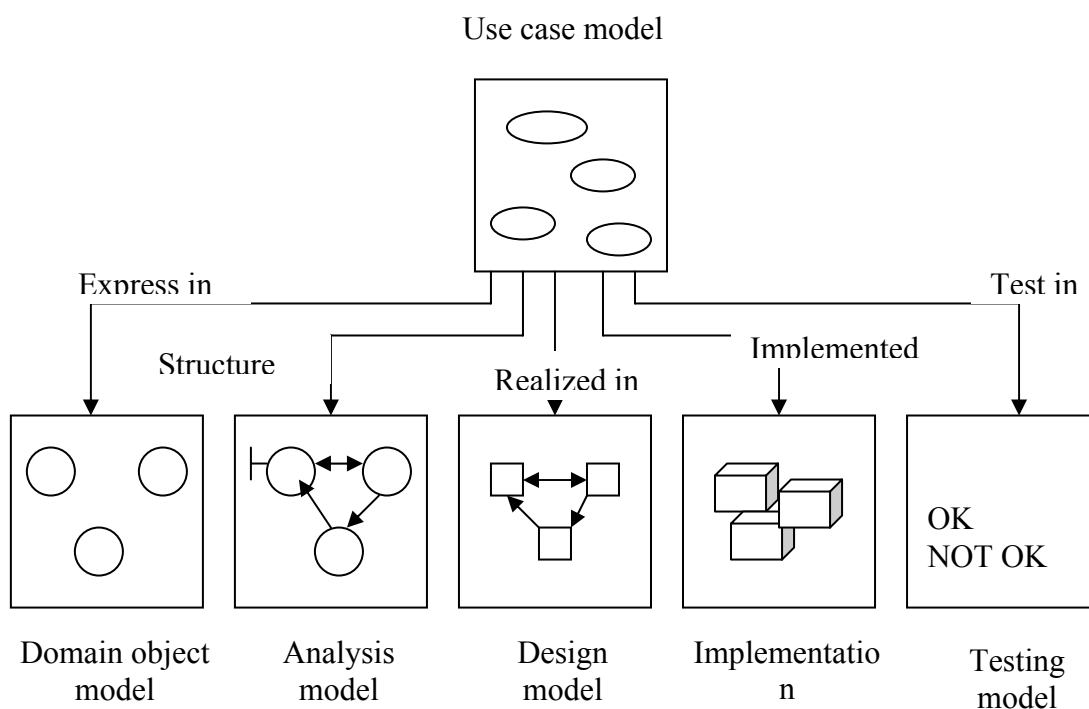
2.5.2 The Booch Methodology เป็นที่นิยมใช้กันอย่างกว้างขวางโดยวิธีเชิงวัตถุซึ่งช่วยให้ออกแบบระบบที่ใช้วัตถุ เป็นแบบอย่างได้ โดยครอบคลุมไปถึงการวิเคราะห์และการออกแบบ วิธีการของ Booch ประกอบไปด้วย ไดอะแกรมต่างๆต่อไปนี้

- คลาสไดอะแกรม
- ออบเจกต์ไดอะแกรม
- สเตททรานซิชันไดอะแกรม
- โมดูลไดอะแกรม
- โพเชสไดอะแกรม
- อินเตอร์แอกชันไดอะแกรม

2.5.3 The Jacobson et al. Methodologies หัวใจหลักของพัฒนาระบบ วิธีนี้คือ หลักการซึ่งทำให้เกิด Objectory ขึ้นมา

2.5.3.1 Object Oriented Software Engineering: เรียกว่า Objectory ซึ่งเป็นวิธีการพัฒนาระบบด้วยวิธีเชิงวัตถุ ซึ่งสร้างโมเดลที่แตกต่างกันดังนี้

- ยูสเคสโมเดล (Use case Model) ซึ่งกำหนดสิ่งที่มากระทำกับระบบ (Actors) และมาใช้ระบบอย่างไร
- โดเมนออบเจกต์โมเดล (Domain object Model) เป็นการแมปออบเจกต์ที่มีอยู่จริงให้เป็นไปตามโดเมนของระบบ
- โมเดลการวิเคราะห์ (Analysis object Model) เป็นการนำเสนอว่าควรสร้างระบบอย่างไร มีการส่งข้อมูลอย่างไรบ้าง
- โมเดลของการสร้าง (Implementation model) เป็นการแสดงการสร้างระบบ
- โมเดลการทดสอบ (Test model) เป็นการวางแผนการทดลองระบบที่สร้างขึ้น และทำรายงานต่างๆ

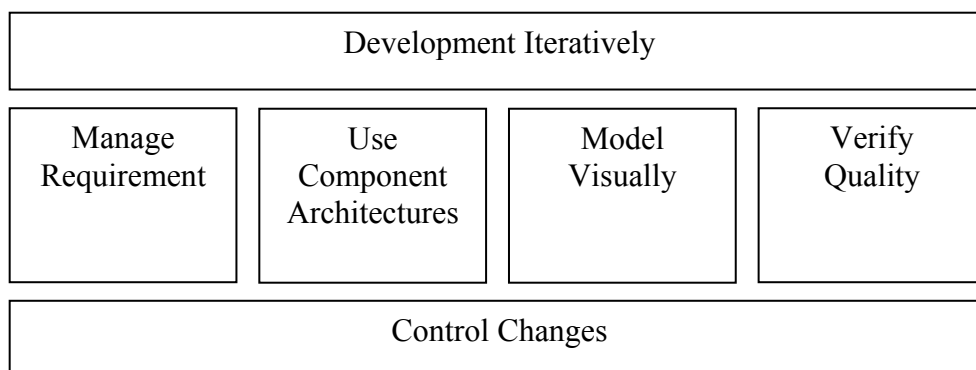


รูปที่ 2-1 การเชื่อมโยงของ Use Case Model กับ Model อื่น ๆ

2.5.3.2 Object Oriented Business Engineering (OOBE) เป็นวิธีการเชิงวัตถุที่มีระดับความเสี่ยง โดยใช้ยูสเคสเป็นตัวหลักในการทำโมเดลต่างๆ ตาม Software engineering process โดยแบ่งเป็นขั้นตอนต่างๆ ดังนี้

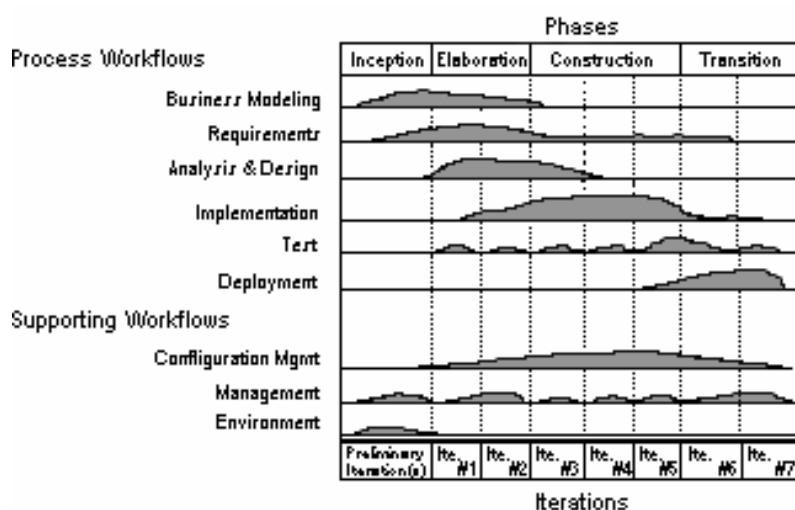
- Analysis phase เป็นการวิเคราะห์และกำหนดว่าระบบสร้างขึ้นมาเพื่อทำอะไร ตามโมเดลของโดเมนปัญหาของระบบ (Domain object model), โมเดลของความต้องการ (Requirement model) และ โมเดลการวิเคราะห์ (analysis model)
- Design and implementation phase เป็นการกำหนดสิ่งที่จำเป็นต่างๆในโมเดลของการออกแบบ (Design model) ซึ่งจะรวมแฟกเตอร์ต่างๆไว้ เช่น ระบบการจัดการฐานข้อมูล (DBMS), การกระจายโปรเซสการเขียนโปรแกรมการใช้เครื่องมือ (Tools) ต่างๆ เป็นต้น
- Testing phase สุดท้ายเป็นการทดสอบระบบโดย Jacobson ได้แสดงไว้หลายวิธีในระดับต่างๆ คือการทดสอบในระดับยูนิต (unit testing), การทดสอบในระดับการทำงานร่วมกัน (Integrate testing) และ การทดสอบระบบ(system testing)

2.6 กระบวนการพัฒนาซอฟต์แวร์(Software Process) ที่ดี



รูปที่ 2-2 *Best Practices of Software Engineering*

2.6.1 การพัฒนาแบบวนรอบ (Development Iteratively) จากรูปจะเห็นได้ว่า ส่วนนี้จะนำไปใช้กับส่วนของ Manage Requirement ,Use Component Architecture ,Model Visually และ Verify Quality ดังนั้นการพัฒนาแบบวนรอบนี้ ถือว่าเป็นกระบวนการที่สำคัญกระบวนการหนึ่งทีเดียว คุณลักษณะของการพัฒนาแบบวนรอบคือ นักพัฒนาจะสามารถวิเคราะห์ และแก้ปัญหาที่มีความเสี่ยงสูง ก่อนที่จะมีการลงทุนพัฒนาระบบ การพัฒนาแบบวนรอบสามารถได้รับความต้องการอย่างต่อเนื่องจากผู้ใช้ได้ นอกจากนั้น การทดสอบและการรวมระบบจะค่อยเป็นค่อยไปอย่างต่อเนื่อง การมี milestone ทำให้นักพัฒนาทราบว่าตอนนี้การพัฒนากำลังเน้นไปที่จุดใด



รูป 2-3 *Deployment Iteratively Process*

จากรูป แสดงให้เห็นถึงการประยุกต์ใช้การพัฒนาแบบวนรอบ สำหรับในการพัฒนารอบแรกนั้นจะใช้เวลากับกระบวนการ Business Modeling มากที่สุด ในรอบต่อไปก็จะใช้เวลาการศึกษา Requirement มากที่สุด แต่ในทุกกรอบการทำงานก็จะทำในกระบวนการทุกกระบวนการ แต่จะมากน้อยแตกต่างกันไปตามรอบของการพัฒนา

2.6.2 การจัดการความต้องการของระบบ (Requirement Management) เป็นกระบวนการที่เป็นระบบในการค้นหา จัดโครงสร้าง และทำเอกสารความต้องการของระบบ และเป็นกระบวนการในการจัดตั้ง ดูแลข้อตกลงระหว่างลูกค้า หรือผู้ใช้ กับทีมพัฒนาของโครงการในการเปลี่ยนแปลงความต้องการของระบบ

2.6.3 การใช้สถาปัตยกรรมคอมโพเน้น (Use Component Architectures) สถาปัตยกรรมซอฟต์แวร์เป็นโครงสร้างหลักของระบบซอฟต์แวร์และเป็นส่วนหนึ่งที่ได้จากการออกแบบระบบซึ่งเกี่ยวข้องกับโครงสร้างและพฤติกรรมของระบบในด้านของการใช้งาน หน้าที่การทำงาน ประสิทธิภาพ และเงื่อนไขของเทคโนโลยี กระบวนการนี้ จะทำให้สถาปัตยกรรมซอฟต์แวร์มีพื้นฐานอยู่บน หลักการของ Component เป็นหลัก ซึ่งจะทำให้สถาปัตยกรรมที่ได้นั้นมีความยืดหยุ่นและทำให้สามารถดูแลรักษา เพิ่มเติมระบบได้ง่าย และยังสามารถนำกลับมาใช้ได้

2.6.4 แบบจำลองรูปภาพ (Model Visually) - เป็นการนำเอาโมเดลที่เป็นรูปภาพมาช่วย ซึ่งทำให้สามารถเก็บโครงสร้างและพฤติกรรมของสถาปัตยกรรม แสดงส่วนประกอบในระบบ หรือแสดงรายละเอียดของงาน โดยมีการดูแลรักษาความตรงกันของการออกแบบและการพัฒนา และยังช่วยลดความกำกวมในการสื่อสาร ส่วนนี้จะเป็นส่วนที่เราจะพูดถึง UML กัน ซึ่งรายละเอียดจะกล่าวต่อไป

2.6.5 การทดสอบคุณสมบัติ (Verify Quality) การพัฒนาแบบวนรอบนั้นทำให้นักพัฒนาสามารถทดสอบระบบได้ดียิ่งขึ้น ทั้งนี้แทนที่จะทำการทดสอบระบบครั้งใหญ่ครั้งเดียวก่อนที่จะส่งให้กับผู้ใช้ ด้วยการพัฒนาแบบวนรอบจะทำให้มีการทยอยทดสอบในแต่ละรอบ ส่งผลให้ระบบในรอบต่อไปมีคุณภาพมากยิ่งขึ้น

2.6.6 การควบคุมความเปลี่ยนแปลง (Control Changes) หลักการในการจัดการความเปลี่ยนแปลงก็คือจะต้องมีการแยกสถาปัตยกรรมของระบบออกเป็นระบบย่อยๆ และมีการกำหนดความรับผิดชอบของระบบย่อยให้กับทีมพัฒนาหนึ่ง มีการจัดตั้งพื้นที่ทำงานสำหรับนักพัฒนาแต่ละคน มีการกำหนดการรวมกันของแต่ละพื้นที่ทำงาน มีการกำหนดวิธีการควบคุมความเปลี่ยนแปลง มีการบันทึกความเปลี่ยนแปลงในแต่ละรุ่น มีการทดสอบในแต่ละรอบของการพัฒนาระบบ

ระบบที่ถูกต้องนั้น จะมีการจัดการความต้องการความต้องการพื้นฐาน จากนั้นจึงเปรียบเทียบ ความเปลี่ยนแปลงที่เกิดขึ้นโดยใช้สถาปัตยกรรมคอมโพเน้น นักพัฒนาสามารถที่จะตรวจสอบรุ่นของ คอมโพเน้นที่จะใช้โมเดลด้วยรูปภาพ ทำให้สามารถตรวจสอบความเปลี่ยนแปลงได้ง่าย ผลที่ได้จากการตรวจสอบจะต้องสามารถติดตามได้ในแต่ละรุ่นของระบบ จากนั้นจะพัฒนานวนรอบต่อไป

จากรายละเอียดที่ได้กล่าวข้างต้น การสื่อสารระหว่างนักพัฒนา นับเป็นปัญหาหนึ่งในการพัฒนาระบบซอฟต์แวร์ใหญ่ๆ ดังนั้นในกระบวนการการพัฒนาซอฟต์แวร์ที่ได้นั้นจึงนำเอา แบบจำลองรูปภาพ (Visual Modeling) มาใช้ในการแสดงรายละเอียดของงาน แทนที่จะใช้ข้อความในการอธิบายเหมือนระบบในอดีต ซึ่งการใช้ข้อความนั้นอาจจะทำให้ความหมายนั้นกำกวม ต่างกับรูปภาพที่แทนคอมโพเน้น ในระบบจริงๆ ที่สามารถสื่อความหมายได้ชัดเจน

2.6.7 แบบจำลองรูปภาพ (Visual Modeling Technology: VMT) VMT คือแนวทางหนึ่งของการพัฒนาระบบ โดยการออกแบบระบบด้วยโมเดลตามแนวความคิดของโลกแห่งความเป็นจริง โมเดลที่ถูกออกแบบมานั้น จะมีประโยชน์ช่วยในการเข้าใจถึงปัญหาของระบบ, การติดต่อกันของผู้ที่จัดทำโครงการ (เช่น ลูกค้า, ผู้ดูแลระบบ, นักวิเคราะห์ และนักออกแบบ เป็นต้น) , การเตรียมการทำเอกสาร ,การออกแบบโปรแกรมและดาตาเบส

โมเดลเป็นนามธรรมที่แสดงถึงลักษณะที่สำคัญของปัญหาหรือโครงสร้างของระบบ โดยการกลั่นกรองรายละเอียดที่ไม่สำคัญออก ดังนั้นจึงเป็นการทำให้ปัญหานั้นง่ายต่อการเข้าใจ เราจึงควรสร้างโมเดลสำหรับระบบที่ซับซ้อน เพราะความสามารถของคนที่จะเข้าใจระบบที่ซับซ้อนนั้นมีจำกัด การสร้างโมเดลจะทำให้นักออกแบบสามารถเห็นภาพกว้างของโครงการได้ โมเดลที่ดีควรจะใช้สัญลักษณ์ที่ถูกต้องและมีการตรวจสอบโมเดลให้ตรงความต้องการของระบบ

Visual Modeling Technique จะมีส่วนเกี่ยวข้องกับทุกขั้นตอนของการพัฒนาโปรแกรมเชิงวัตถุทั้งการรวบรวมความต้องการ(requirement gathering), การวิเคราะห์ (analysis), การออกแบบ (design), การเขียนโปรแกรม (coding) และการทดสอบ (testing)

2.7 ส่วนประกอบของ UML

UML ประกอบด้วยหลายๆส่วนดังนี้

View : แสดงมุมมองต่างๆ ของระบบที่ออกแบบขึ้นมา โดยการอธิบายรายละเอียดในแต่ละส่วนใน View จะใช้ไคอะแกรม ที่มีอยู่ใน UML อธิบาย

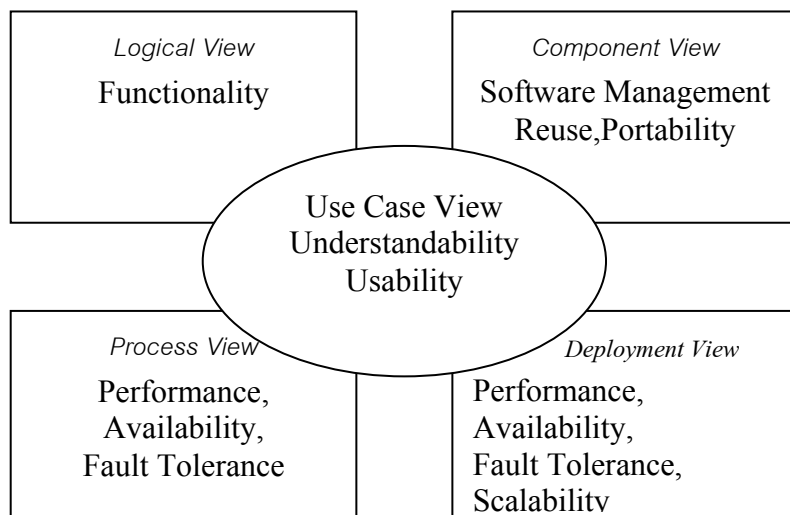
Diagram : เป็นไคอะแกรมที่ใช้ในการอธิบายส่วนต่างๆของ View

Model element : เป็นสัญลักษณ์ที่ใช้ในไคอะแกรมเพื่อแสดงหรือเป็นตัวแทนของสิ่งต่างๆ เช่น คลาส (class) ,ออปเจ็กต์,เมสเสจ(message) และ ความสัมพันธ์(relationship) เป็นต้น

General mechanism : เป็นส่วนที่แสดงคอมเม้นเพิ่มเติม (extra comment) , ข้อมูลอื่นๆ ที่จำเป็นหรือความหมายของโมเดลเอลิเมนต์ (model element)

2.7.1 View ในการออกแบบระบบที่มีขนาดใหญ่และมีความซับซ้อนมากกะนั้นจะทำให้ผู้ออกแบบระบบไม่สามารถที่จะออกแบบระบบได้ครบถ้วน ดังนั้นจึงต้องมีการมองระบบเป็นมุมมองต่างๆ เพื่อทำให้ง่ายในการออกแบบ เช่น มุมมองด้านฟังก์ชันนอล , นอนฟังก์ชันนอล(Nonfunctional), มุมมองขององค์กร เป็นต้น ดังนั้นระบบจึงมี View ที่ต่างๆ กันซึ่งแต่ละ view จะแสดงมุมมองเฉพาะของระบบซึ่งอธิบายรวมกันเป็นระบบที่สมบูรณ์ ซึ่งจะประกอบด้วย View ต่างๆ ดังนี้

2.7.1.1 Use-case View อธิบายการทำงานต่างๆ ของระบบที่ถูกมองจากภายนอกหรือผู้ใช้ระบบ Use-case view ซึ่งอธิบายโดยยูสเคสไคอะแกรม(use-case diagram) เป็นมุมมองสำหรับลูกค้า , ผู้ออกแบบ ,ผู้พัฒนาระบบและผู้ทดสอบระบบ



รูปที่ 2-4 สถาปัตยกรรมของ View

2.7.1.2 Logical view อธิบายการทำงานต่างๆ ที่ถูกออกแบบไว้ภายในระบบ ว่าระบบจะมีบริการอะไรให้กับผู้ใช้งาน โดยจะแสดงโครงสร้างแบบสถิต (Static) เช่น คลาส, ออบเจกต์, ความสัมพันธ์และการทำงานร่วมกันแบบไดนามิก (dynamic collaboration) ซึ่งจะเกิดขึ้นเมื่อออบเจกต์ ส่งเมสเสจ ระหว่างกันในการทำงาน

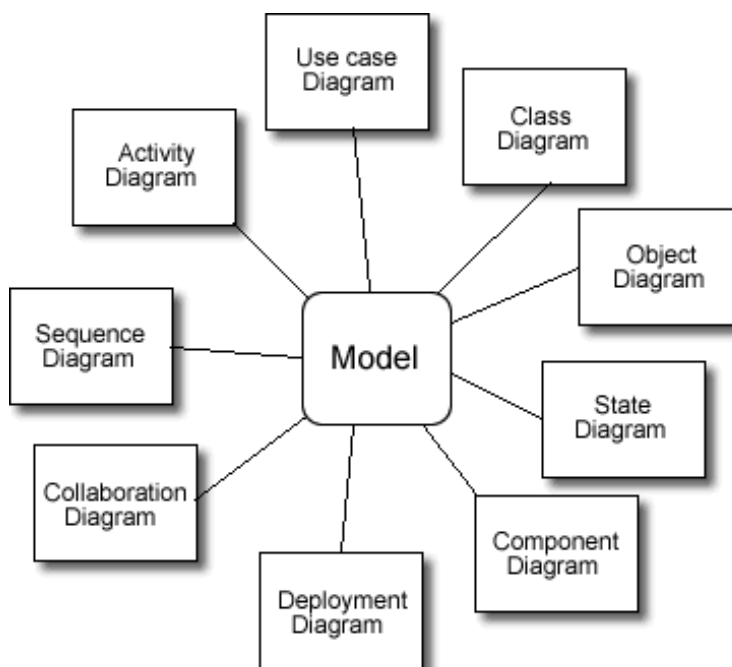
โครงสร้างแบบสถิตจะอธิบายโดยใช้ คลาสไดอะแกรม (Class diagram) และออบเจกต์ไดอะแกรม (object diagram) ส่วนการทำงานร่วมกันแบบไดนามิกจะอธิบายโดยใช้ สเตตไดอะแกรม (state diagram) และแอคทิวิตีไดอะแกรม (activity diagram)

2.7.1.3 Component View อธิบายการสร้างและสร้างความขึ้นต่อกันของโมดูล (Module) ที่ใช้ในการพัฒนาระบบ โดยใช้คอมโพเนนต์ไดอะแกรม (Component diagram) ในการอธิบาย

2.7.1.4 Deployment View อธิบายการจัดวางระบบให้เหมาะในด้านกายภาพ (Physical) แสดงด้วยคอมพิวเตอร์และโหนด (nodes) ต่างๆ เพื่อให้ระบบมีเสถียรภาพมากขึ้น โดยใช้ดีพลอยเมนต์ไดอะแกรม (deployment diagram) ในการอธิบาย

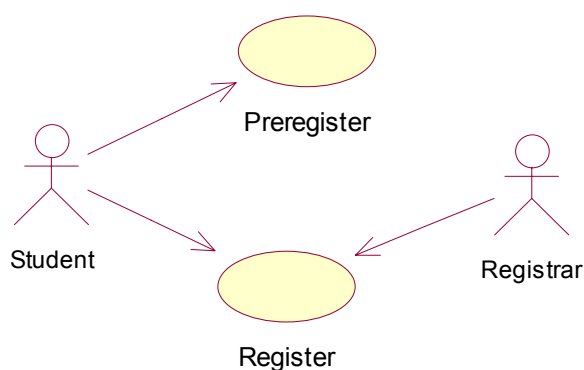
2.7.1.5 Process View แสดงการทำงานร่วมกันและการติดต่อกันของส่วนต่างๆ ในระบบ

2.7.2 Diagram เป็นไคอะแกรมซึ่งแสดงสัญลักษณ์ที่ถูกจัดเรียงเพื่ออธิบายระบบในมุมมองต่างๆ ซึ่งในระบบหนึ่งๆ จะประกอบด้วยหลายๆ ไคอะแกรม และในแต่ละไคอะแกรมยังสามารถมองในหลายๆ มุมมองได้ UML มีไคอะแกรมที่ต่างกันอยู่ ดังนี้



รูปที่ 2-5 แสดงไคอะแกรมต่างๆ ของ UML

2.7.2.1 Use Case Diagram ยูสเคสไคอะแกรมจะแสดงความสัมพันธ์ระหว่างผู้ใช้งาน (User) กับระบบ โดยใช้แอ็กเตอร์ (actors) แทนผู้ใช้งานและแอ็กเตอร์จะต้องติดต่อกับระบบโดยผ่าน ยูสเคส(Use case) ต่างๆ ซึ่งยูสเคสใน Use case diagram ก็คือการทำงานต่างๆ ของระบบที่ผู้ใช้ต้องการ



รูปที่ 2-6 ตัวอย่าง Use Case ในระบบลงทะเบียน

แอ็กเตอร์ไม่ใช่ส่วนประกอบของระบบ แต่จะเป็นส่วนที่ใช้ติดต่อกับระบบ ซึ่งอาจจะเป็นเพียงการป้อนข้อมูลเข้าไปในระบบหรือเป็นเพียงการรับข้อมูลจากระบบ หรืออาจจะเป็นทั้งสองอย่างก็ได้ ในระบบหนึ่งๆ จะต้องมียแอ็กเตอร์ซึ่งจะต้องทำการกำหนดคุณลักษณะของแอ็กเตอร์ (Identify actor) ให้ได้ก่อนดังนี้

- ใครเป็นผู้ใช้งานระบบ?
- ใครที่มีความเหมาะสมที่จะใช้งานระบบ?
- ใครที่มีส่วนสนับสนุนข้อมูลในระบบ ใช้ข้อมูลในระบบและแก้ไขข้อมูลในระบบ?
- ใครที่สนับสนุนการบำรุงรักษาระบบ (Maintenance)?
- ระบบมีการใช้งานกับภายนอกหรือเปล่า?

2.7.2.2 Class Diagram คลาสไดอะแกรมเป็นไดอะแกรมที่มีความสำคัญมากที่สุดในการวิเคราะห์และออกแบบโดยใช้วิธีเชิงวัตถุ เนื่องจากคลาสดิอะแกรมจะแสดงโครงสร้างของวัตถุและคลาสที่มีในระบบรวมทั้งแสดงความสัมพันธ์ด้วย คลาสดิอะแกรมจะเป็นโครงสร้างหลักของระบบและยังใช้ในการแยกย่อยรายละเอียด (Decomposite) ด้วยวิธีเชิงวัตถุ

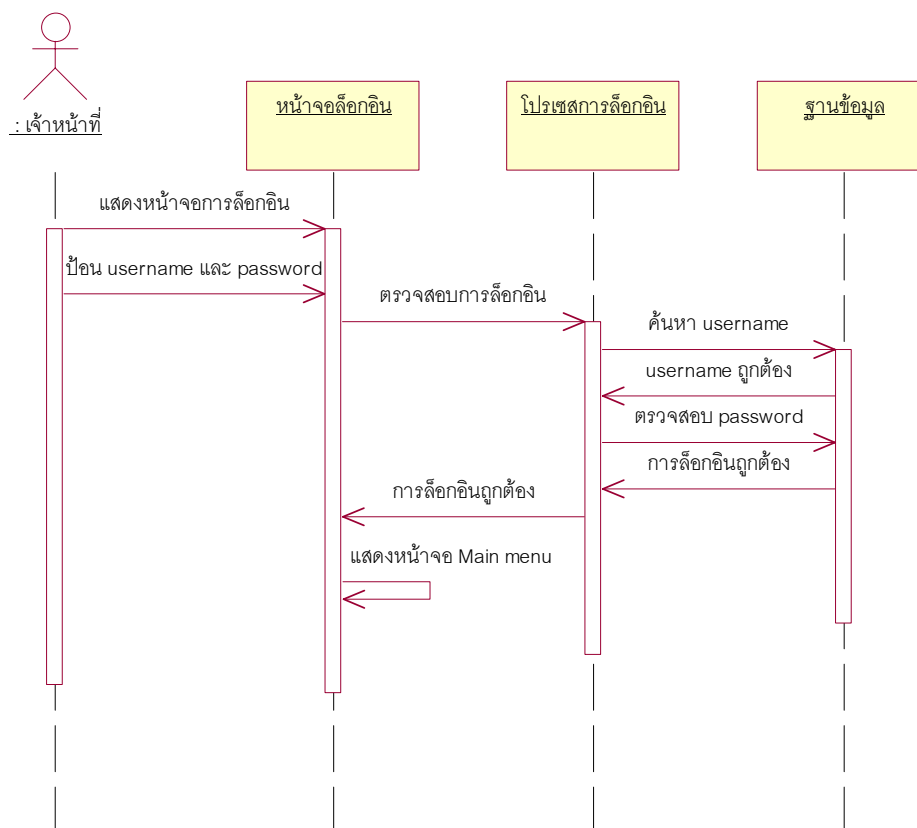
สัญลักษณ์ความสัมพันธ์ต่างๆ ของคลาสดิอะแกรม มีดังนี้

- 1 หมายถึงจะมีออบเจกต์ในคลาสดิอะแกรมได้หนึ่งออบเจกต์เท่านั้น
- 0..1 หมายถึงจะมีออบเจกต์ในคลาสดิอะแกรมได้แก่หนึ่งอาจจะไม่มีก็ได้
- M..N หมายถึงจะมีออบเจกต์ในคลาสดิอะแกรมได้ตั้งแต่ M ถึง N (เมื่อ M และ N เป็นจำนวนเต็มบวก)
- * หมายถึงจะมีออบเจกต์ในคลาสดิอะแกรมได้ตั้งแต่ศูนย์ขึ้นไป
- 0..* หมายถึงจะมีออบเจกต์ในคลาสดิอะแกรมได้ตั้งแต่ศูนย์ขึ้นไป
- 1..* หมายถึงจะมีออบเจกต์ในคลาสดิอะแกรมได้ตั้งแต่หนึ่งขึ้นไป

2.7.2.3 Object Diagram ออบเจกต์ไดอะแกรมจะได้อมาจากคลาสดิอะแกรม ซึ่งจะเป็นการแสดงการเชื่อมต่อระหว่างอินชแทนซ์ (Instances) ต่างๆ ที่อยู่ภายในคลาสดิอะแกรมอย่างละเอียด เพื่อใช้เป็นตัวอย่างสำหรับคลาสดิอะแกรมที่มีความซับซ้อนโดยจะนำมาแทนเป็น ออบเจกต์จริงๆ และแสดงความสัมพันธ์ต่างๆ ให้เห็น โดยสัญลักษณ์ของอินชแทนซ์ในออบเจกต์ไดอะแกรม จะใช้การขีดเส้นใต้เป็นสัญลักษณ์ ภายในกรอบสี่เหลี่ยม

2.7.2.4 Collaboration Diagram คีอลแลโบเรชั่นไดอะแกรมใช้อธิบายการทำงานร่วมกันของออบเจกต์ที่มีการส่งแอสเซสระหว่างกัน เพื่อบอกถึงลำดับขั้นตอนการทำงานของระบบ โดยใช้สัญลักษณ์ลูกศรเป็นตัวกำหนดทิศทางซึ่ง คีอลแลโบเรชั่นไดอะแกรมจะเป็นส่วนอธิบายการทำงานของออบเจกต์ไดอะแกรม

2.7.2.5 Sequence Diagram ซีเควินซ์ไดอะแกรมแสดงการติดต่อโต้ตอบระหว่างออบเจ็กต์ ที่เน้นไปที่ลำดับก่อนหลังและเวลาที่มีความสำคัญในการเปลี่ยนแปลง จะมีการส่งแมสเสจ เพื่อให้ออบเจ็กต์ ต่างๆ ที่อยู่ในระบบทำงานโดยใช้สัญลักษณ์ที่เรียกว่า ออบเจ็กต์แมสเสจ ซีเควินซ์ชาท (Object message sequence chart) ในการแสดงดังรูป

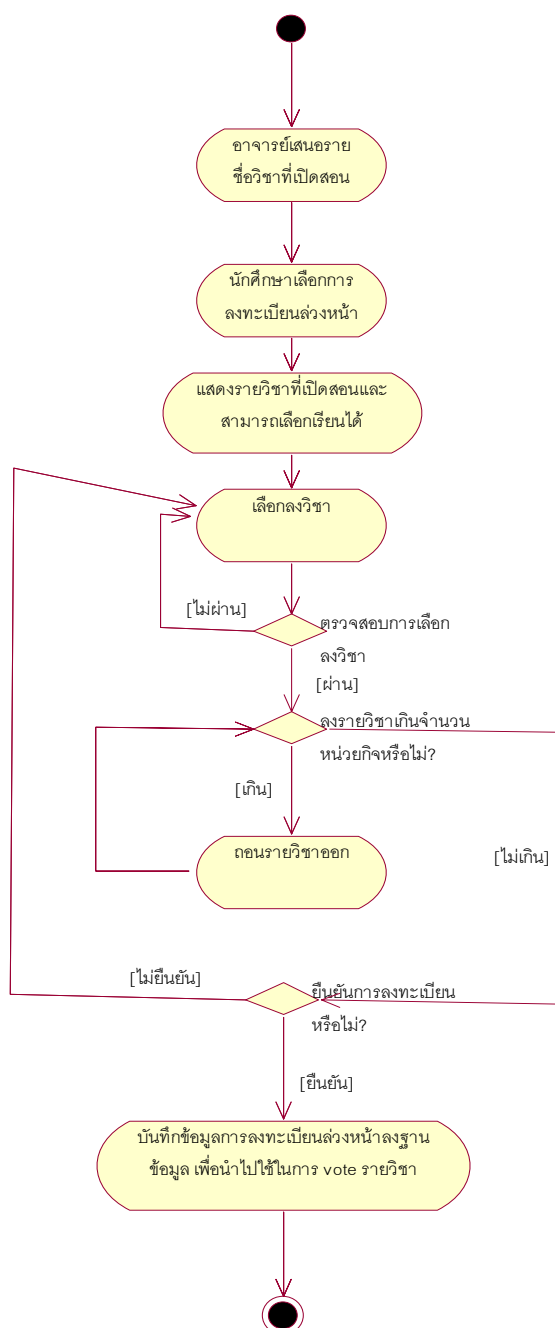


รูปที่ 2-7 แสดงตัวอย่าง ซีเควินซ์ไดอะแกรม

2.7.2.6 State Diagram สเตทไดอะแกรมใช้อธิบายคลาสต่างๆ ในระบบโดยจะแสดงทุกๆ สถานะที่เป็นไปได้และเหตุการณ์ทำให้ออบเจ็กต์เหล่านั้นเกิดการเปลี่ยนแปลง โดยเหตุการณ์ที่ทำให้เกิดการเปลี่ยนแปลงอาจเกิดจากออบเจ็กต์อื่นส่งแมสเสจมา การเปลี่ยนแปลงสถานะเรียกว่า ทรานซิชัน (Transitions)

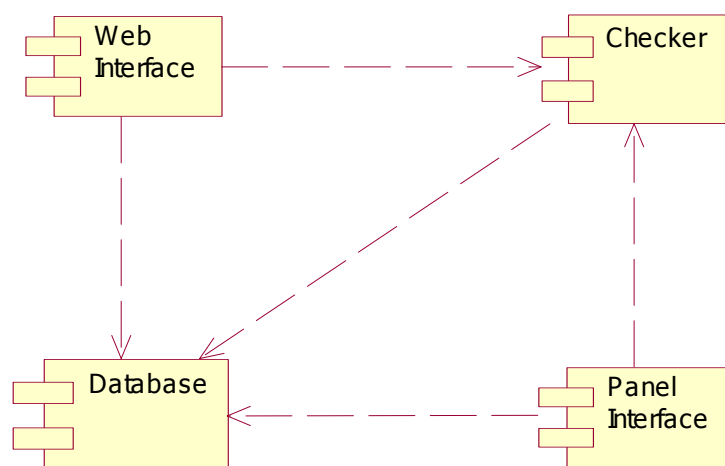
2.7.2.7 Activity diagram แอคติวิตี้ไคอะแกรมแสดงลำดับการไหลของกิจกรรม(activity)

ต่างๆ โดยจะอธิบายกิจกรรมต่างๆ ในลักษณะของการกระทำในไคอะแกรม จะแสดงเป็นสถานะการกระทำ (Action State) ซึ่งสถานะจะเปลี่ยนไปเมื่อเกิดการกระทำอย่างใดอย่างหนึ่งเกิดขึ้นตามเงื่อนไขหรือการตัดสินใจที่กำหนดไว้เพื่อควบคุมการไหลของกิจกรรมรวมถึงสามารถมีแมสเชสที่รับ – ส่งระหว่างแต่ละกิจกรรม



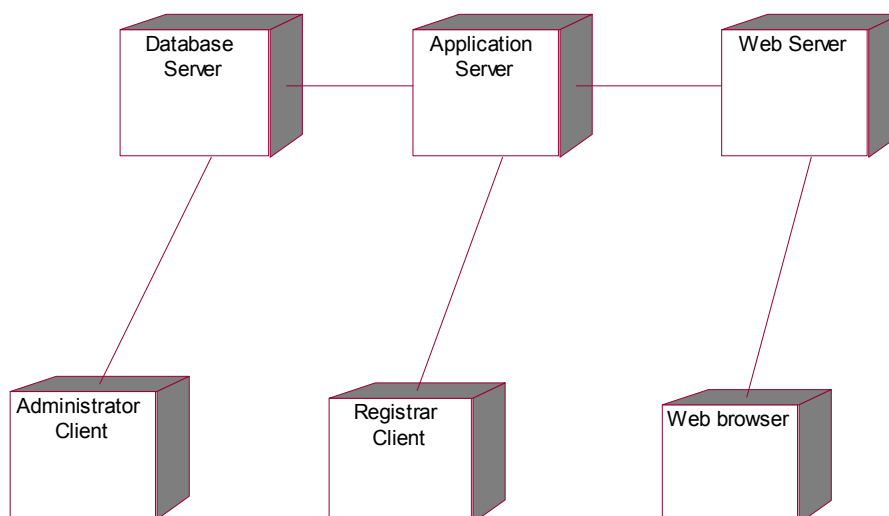
รูปที่ 2-8 แสดงตัวอย่างแอคติวิตี้ไคอะแกรม

2.7.2.8 Component diagram คอมโพเน้นไคอะแกรมแสดงโครงสร้างทางกายภาพของโค้ดในรูปคอมโพเน้นท์ของโค้ท (Code) อาจเป็นส่วนประกอบของ ซอสเมื่อเกิดการเปลี่ยนแปลงของ คอมโพเน้นท์หนึ่งจะมีผลต่อคอมโพเน้นท์อื่นๆ อย่างไร ซึ่งช่วยในการโปรแกรม โค้ด (source code) คอมโพเน้นท์จะประกอบด้วยข้อมูลเกี่ยวกับ ลอจิคัล คลาส (logical class) ของคอมโพเน้นท์ในไคอะแกรม มีการแสดงความสัมพันธ์หรือความพึ่งพากันของคอมโพเน้นท์ให้ช่วยในการวิเคราะห์ว่า



รูปที่ 2-9 แสดงตัวอย่างคอมโพเน้นท์ไคอะแกรม

2.7.2.9 Deployment diagram ดีพลอยเม้นไคอะแกรมแสดงสถาปัตยกรรมทางกายภาพของส่วนประกอบต่างๆ ของฮาร์ดแวร์(hardware) ซึ่งถูกเรียกว่า โหนด(Node) ในระบบซึ่งสามารถแสดงเป็นโหนดและการเชื่อมต่อระหว่างชนิดของการเชื่อมต่อ นอกจากนี้ภายในโหนด ยังสามารถมีคอมโพเน้นท์ หรือออบเจกต์ที่สามารถปฏิบัติกับโหนดเพื่อแสดงว่าโปรแกรมส่วนใดถูกปฏิบัติบนโหนดใดและความสัมพันธ์ระหว่างคอมโพเน้นท์ที่อยู่บนโหนด ซึ่งโหนดจะถูกแทนด้วยสัญลักษณ์ของสี่เหลี่ยมลูกบาศก์ดังรูป



รูปที่ 2-10 แสดงตัวอย่างดีพลอยเม้นไคอะแกรม